

Trabalho Prático de Estrutura de Dados

Sistema de Gerenciamento de Biblioteca com TADs, Lista, Fila e Árvore
Binária de Busca

Disciplina: Estrutura de Dados - UECe

Professor: Bonfim Amaro Junior

Equipe: No máximo, 3 Alunos

1 Tema do Trabalho

O trabalho consistirá no desenvolvimento de um **Sistema de Gerenciamento de Biblioteca**, implementado em linguagem C, utilizando os conceitos de **Tipos Abstratos de Dados – TADs**, estruturas lineares e estrutura não linear.

O sistema deverá permitir o cadastro, a consulta, a listagem, o empréstimo e a devolução de livros. Para isso, os alunos deverão implementar manualmente as estruturas de dados necessárias, sem utilizar bibliotecas prontas para armazenamento dinâmico.

A estrutura principal de armazenamento será uma **Árvore Binária de Busca – ABB**, responsável por organizar os livros a partir de um código identificador. Além disso, o sistema deverá utilizar uma **fila** para gerenciar reservas de livros e uma **lista encadeada** para registrar o histórico de empréstimos realizados.

2 Contextualização do Problema

Uma biblioteca precisa controlar seu acervo de livros, permitindo o cadastro de novas obras, a busca rápida por código, a listagem ordenada dos livros e o controle de empréstimos.

Cada livro possui um código único, título, autor, ano de publicação, quantidade total de exemplares e quantidade disponível para empréstimo.

Quando um livro está disponível, ele pode ser emprestado imediatamente. Quando não há exemplares disponíveis, o sistema deve permitir que o usuário entre em uma fila de reserva. Assim que o livro for devolvido, o primeiro usuário da fila de reservas poderá ser atendido.

Além disso, o sistema deverá manter um histórico dos empréstimos realizados, permitindo visualizar quais livros foram emprestados e para quais usuários.

3 Objetivo Geral

Desenvolver uma aplicação em linguagem C para gerenciamento de uma biblioteca, utilizando Tipos Abstratos de Dados, lista encadeada, fila e Árvore Binária de Busca.

4 Objetivos Específicos

O trabalho deverá permitir que o aluno seja capaz de:

- implementar Tipos Abstratos de Dados em linguagem C;
- organizar o código em arquivos `.h` e `.c`;
- manipular ponteiros e alocação dinâmica de memória;
- implementar uma lista encadeada para registro de histórico;
- implementar uma fila para controle de reservas;
- implementar uma Árvore Binária de Busca para armazenamento e consulta dos livros;
- integrar diferentes estruturas de dados em uma aplicação funcional;
- realizar testes sobre as operações implementadas;
- apresentar e defender tecnicamente a solução desenvolvida.

5 Descrição Geral da Aplicação

A aplicação deverá funcionar em modo texto, com menu interativo, permitindo ao usuário realizar operações sobre o acervo da biblioteca.

O sistema deverá trabalhar com três entidades principais:

1. **Livro:** representa uma obra cadastrada na biblioteca.
2. **Reserva:** representa um usuário aguardando por um livro sem exemplares disponíveis.
3. **Empréstimo:** representa o registro de um empréstimo realizado.

6 Estruturas Obrigatórias

6.1 TAD Livro

O aluno deverá implementar um Tipo Abstrato de Dados para representar um livro.

Cada livro deverá conter, no mínimo:

```
1 int codigo;  
2 char titulo[100];  
3 char autor[100];  
4 int ano;  
5 int quantidadeTotal;  
6 int quantidadeDisponivel;
```

O TAD Livro deverá possuir funções como:

```
1 Livro* criarLivro(int codigo, char titulo[], char autor[],  
2                   int ano, int quantidadeTotal);  
3  
4 void exibirLivro(Livro* livro);  
5  
6 int obterCodigoLivro(Livro* livro);  
7  
8 int obterQuantidadeDisponivel(Livro* livro);  
9  
10 void emprestarExemplar(Livro* livro);  
11  
12 void devolverExemplar(Livro* livro);
```

6.2 Árvore Binária de Busca de Livros

A Árvore Binária de Busca será a principal estrutura de armazenamento do acervo.

A árvore deverá organizar os livros pelo campo `codigo`. Cada nó da árvore deverá armazenar um ponteiro para um livro.

Estrutura sugerida:

```
1 typedef struct NoArvore {  
2     Livro* livro;  
3     struct NoArvore* esquerda;  
4     struct NoArvore* direita;  
5 } NoArvore;  
6  
7 typedef struct Arvore {  
8     NoArvore* raiz;
```

```
9 } Arvore;
```

Operações obrigatórias da ABB:

```
1 Arvore* criarArvore();
2
3 void inserirLivroArvore(Arvore* arvore, Livro* livro);
4
5 Livro* buscarLivroArvore(Arvore* arvore, int codigo);
6
7 void listarLivrosEmOrdem(Arvore* arvore);
8
9 void listarLivrosPreOrdem(Arvore* arvore);
10
11 void listarLivrosPosOrdem(Arvore* arvore);
12
13 int contarLivros(Arvore* arvore);
14
15 int calcularAlturaArvore(Arvore* arvore);
```

A operação de remoção de livros da árvore poderá ser considerada requisito extra, mas não obrigatória.

6.3 Fila de Reservas

A fila deverá ser utilizada para controlar os usuários que desejam reservar um livro quando não há exemplares disponíveis.

Cada reserva deverá conter:

```
1 char nomeUsuario[100];
2 int codigoLivro;
```

A fila deverá seguir a política **FIFO – First In, First Out**, isto é, o primeiro usuário a entrar na fila deverá ser o primeiro a ser atendido.

Estrutura sugerida:

```
1 typedef struct Reserva {
2     char nomeUsuario[100];
3     int codigoLivro;
4 } Reserva;
5
6 typedef struct NoFila {
7     Reserva reserva;
8     struct NoFila* proximo;
```

```

9 } NoFila;
10
11 typedef struct Fila {
12     NoFila* inicio;
13     NoFila* fim;
14 } Fila;

```

Operações obrigatórias da fila:

```

1 Fila* criarFila();
2
3 void enfileirarReserva(Fila* fila, Reserva reserva);
4
5 Reserva desenfileirarReserva(Fila* fila);
6
7 int filaVazia(Fila* fila);
8
9 void exibirReservas(Fila* fila);

```

A fila poderá ser única para todas as reservas, desde que cada reserva armazene o código do livro correspondente.

6.4 Lista Encadeada de Histórico de Empréstimos

A lista encadeada deverá armazenar o histórico de empréstimos realizados.

Cada registro de empréstimo deverá conter:

```

1 char nomeUsuario[100];
2 int codigoLivro;
3 char tituloLivro[100];

```

Estrutura sugerida:

```

1 typedef struct Emprestimo {
2     char nomeUsuario[100];
3     int codigoLivro;
4     char tituloLivro[100];
5 } Emprestimo;
6
7 typedef struct NoLista {
8     Emprestimo emprestimo;
9     struct NoLista* proximo;
10 } NoLista;
11
12 typedef struct Lista {

```

```
13     NoLista* inicio;  
14 } Lista;
```

Operações obrigatórias da lista:

```
1 Lista* criarLista();  
2  
3 void inserirEmprestimo(Lista* lista, Emprestimo emprestimo);  
4  
5 void listarEmprestimos(Lista* lista);  
6  
7 int listaVazia(Lista* lista);
```

7 Funcionalidades Obrigatórias do Sistema

O sistema deverá possuir um menu principal com, no mínimo, as seguintes opções:

1. Cadastrar novo livro
2. Buscar livro por código
3. Listar livros em ordem crescente de código
4. Listar livros em pre-ordem
5. Listar livros em pos-ordem
6. Realizar empréstimo de livro
7. Devolver livro
8. Exibir fila de reservas
9. Exibir histórico de empréstimos
10. Exibir quantidade de livros cadastrados
11. Exibir altura da árvore
0. Sair

8 Regras de Funcionamento

8.1 Cadastro de Livro

Ao cadastrar um novo livro, o sistema deverá solicitar:

- código;
- título;
- autor;
- ano de publicação;

- quantidade total de exemplares.

O livro deverá ser inserido na Árvore Binária de Busca.

Não deverão ser aceitos dois livros com o mesmo código.

8.2 Busca de Livro

O sistema deverá permitir a busca de um livro a partir do seu código.

Caso o livro exista, suas informações deverão ser exibidas.

Caso não exista, o sistema deverá informar que o livro não foi encontrado.

8.3 Listagem de Livros

A listagem em ordem deverá exibir os livros em ordem crescente de código.

Além disso, o sistema deverá apresentar os percursos:

- pré-ordem;
- pós-ordem.

Esses percursos deverão ser usados para demonstrar o funcionamento da Árvore Binária de Busca.

8.4 Empréstimo de Livro

Para realizar um empréstimo, o sistema deverá solicitar:

- nome do usuário;
- código do livro.

O sistema deverá verificar se o livro existe na árvore.

Se o livro existir e houver exemplar disponível, o sistema deverá:

- reduzir a quantidade disponível do livro;
- registrar o empréstimo na lista de histórico;
- exibir mensagem confirmando o empréstimo.

Se o livro existir, mas não houver exemplar disponível, o sistema deverá:

- oferecer a opção de inserir o usuário na fila de reservas;
- enfileirar a reserva caso o usuário aceite.

Se o livro não existir, o sistema deverá exibir uma mensagem de erro.

8.5 Devolução de Livro

Para realizar uma devolução, o sistema deverá solicitar o código do livro.

Se o livro existir, o sistema deverá aumentar a quantidade disponível.

Caso exista reserva na fila para aquele livro, o sistema deverá informar que há usuários aguardando.

Como versão mínima, o sistema poderá apenas exibir a fila de reservas. Como versão mais completa, poderá atender automaticamente o primeiro usuário da fila referente àquele livro.

8.6 Histórico de Empréstimos

Toda vez que um empréstimo for realizado com sucesso, o sistema deverá registrar a operação na lista encadeada de histórico.

O histórico deverá exibir:

- nome do usuário;
- código do livro;
- título do livro.

9 Organização Obrigatória dos Arquivos

O projeto deverá ser organizado em módulos.

Estrutura mínima sugerida:

```
/trabalho_biblioteca
|
|-- main.c
|-- livro.h
|-- livro.c
|-- arvore.h
|-- arvore.c
|-- fila.h
|-- fila.c
|-- lista.h
|-- lista.c
|-- README.md
```


9.1 Descrição dos Arquivos

main.c Deve conter o menu principal e a integração entre as estruturas.

livro.h e **livro.c** Devem conter a definição e implementação do TAD Livro.

arvore.h e **arvore.c** Devem conter a implementação da Árvore Binária de Busca.

fila.h e **fila.c** Devem conter a implementação da fila de reservas.

lista.h e **lista.c** Devem conter a implementação da lista de histórico de empréstimos.

README.md Deve conter instruções de compilação e execução do programa.

10 Entregáveis Obrigatórios

A equipe deverá entregar os seguintes itens.

10.1 Código-fonte Completo

Deverão ser entregues todos os arquivos `.c` e `.h` necessários para compilar e executar o sistema.

O código deverá estar organizado, comentado e funcional.

10.2 Arquivo README.md

O arquivo deverá conter:

- nome da equipe;
- integrantes;
- descrição resumida do sistema;
- instruções para compilação;
- instruções para execução;
- exemplos de uso;
- observações sobre funcionalidades implementadas;
- limitações conhecidas, caso existam.

Exemplo de instrução de compilação:

```
gcc main.c livro.c arvore.c fila.c lista.c -o biblioteca
```

Exemplo de execução:

```
./biblioteca
```

10.3 Relatório Técnico

O relatório deverá ser entregue em PDF e conter:

1. **Capa:** instituição, curso, disciplina, professor, título do trabalho, nome dos integrantes e data.
2. **Introdução:** apresentação do problema, importância do uso de estruturas de dados e visão geral da solução desenvolvida.
3. **Descrição dos TADs:** explicação do TAD Livro, do TAD Reserva e do TAD Empréstimo.
4. **Descrição da Árvore Binária de Busca:** estrutura utilizada, critério de ordenação, operações implementadas, exemplo de inserção e exemplo de percurso em ordem.
5. **Descrição da Fila:** finalidade da fila no sistema, funcionamento da política FIFO e operações implementadas.
6. **Descrição da Lista Encadeada:** finalidade da lista no histórico de empréstimos e operações implementadas.
7. **Descrição das funcionalidades:** cadastro, busca, listagem, empréstimo, devolução, reservas e histórico.
8. **Análise de complexidade:** busca na ABB, inserção na ABB, percurso em ordem, inserção na fila, remoção da fila e inserção na lista.
9. **Testes realizados:** exemplos de entradas, exemplos de saídas, casos de teste com livro disponível, casos de teste com livro indisponível e casos de busca por livro inexistente.
10. **Conclusão:** principais aprendizados, dificuldades encontradas e melhorias futuras.

10.4 Apresentação Final

A equipe deverá realizar uma apresentação final demonstrando o sistema desenvolvido.

A apresentação deverá conter:

- explicação do problema;

- justificativa das estruturas utilizadas;
- demonstração do código;
- execução do sistema;
- exemplos de uso;
- explicação da ABB;
- explicação da fila;
- explicação da lista;
- análise das principais dificuldades;
- participação de todos os integrantes.

A apresentação deverá ter duração sugerida de **10 a 15 minutos por equipe**, podendo ser ajustada pelo professor conforme o tamanho da turma.

11 Critérios de Avaliação

A nota final do trabalho será composta por **100 pontos**, distribuídos da seguinte forma:

Critério	Pontuação
Implementação do TAD Livro e demais registros	10 pontos
Implementação da Árvore Binária de Busca	15 pontos
Implementação da fila de reservas	10 pontos
Implementação da lista de histórico de empréstimos	10 pontos
Integração das estruturas no sistema	10 pontos
Menu, usabilidade e funcionamento geral	5 pontos
Organização do código em módulos <code>.h</code> e <code>.c</code>	5 pontos
Relatório técnico	5 pontos
Apresentação final do trabalho	30 pontos
Total	100 pontos

12 Detalhamento da Pontuação

12.1 Implementação do TAD Livro e Demais Registros – 10 pontos

Item avaliado	Pontuação
Definição correta da estrutura Livro	2 pontos
Função de criação de livro	2 pontos
Funções de exibição e acesso aos dados	2 pontos
Controle correto de quantidade disponível	2 pontos
Organização adequada do TAD em .h e .c	2 pontos

12.2 Implementação da Árvore Binária de Busca – 15 pontos

Item avaliado	Pontuação
Criação correta da estrutura da árvore	2 pontos
Inserção de livros por código	3 pontos
Busca de livro por código	3 pontos
Percurso em ordem	2 pontos
Percurso em pré-ordem	1,5 ponto
Percurso em pós-ordem	1,5 ponto
Cálculo da altura da árvore	1 ponto
Contagem de nós da árvore	1 ponto

12.3 Implementação da Fila de Reservas – 10 pontos

Item avaliado	Pontuação
Criação correta da estrutura da fila	2 pontos
Inserção de reserva na fila	2 pontos
Remoção de reserva da fila	2 pontos
Verificação de fila vazia	1 ponto
Exibição das reservas	1 ponto
Uso coerente da fila no sistema	2 pontos

12.4 Implementação da Lista de Histórico – 10 pontos

Item avaliado	Pontuação
Criação correta da lista encadeada	2 pontos
Inserção de empréstimos na lista	2 pontos
Exibição do histórico	2 pontos

Verificação de lista vazia	1 ponto
Armazenamento correto dos dados do empréstimo	1 ponto
Uso coerente da lista no sistema	2 pontos

12.5 Integração das Estruturas no Sistema – 10 pontos

Item avaliado	Pontuação
Integração entre cadastro e ABB	2 pontos
Integração entre empréstimo e atualização do livro	2 pontos
Integração entre empréstimo e histórico	2 pontos
Integração entre indisponibilidade e fila de reservas	2 pontos
Funcionamento consistente entre as estruturas	2 pontos

12.6 Menu, Usabilidade e Funcionamento Geral – 5 pontos

Item avaliado	Pontuação
Menu claro e funcional	1,5 ponto
Mensagens adequadas ao usuário	1 ponto
Tratamento básico de erros	1 ponto
Sistema compila e executa corretamente	1,5 ponto

12.7 Organização do Código – 5 pontos

Item avaliado	Pontuação
Separação correta em arquivos .h e .c	2 pontos
Código legível e comentado	1 ponto
Nomes adequados de funções e variáveis	1 ponto
Ausência de repetição excessiva de código	1 ponto

12.8 Relatório Técnico – 5 pontos

Item avaliado	Pontuação
Clareza da descrição do problema	1 ponto
Explicação das estruturas utilizadas	1 ponto
Apresentação dos testes	1 ponto

Análise de complexidade	1 ponto
Organização e formatação do documento	1 ponto

13 Apresentação Final – 30 pontos

A apresentação final terá peso de **30% da nota total**.

13.1 Critérios da Apresentação

Item avaliado	Pontuação
Clareza na explicação do problema	4 pontos
Explicação correta dos TADs implementados	4 pontos
Explicação da Árvore Binária de Busca	5 pontos
Explicação da fila de reservas	3 pontos
Explicação da lista de histórico	3 pontos
Demonstração funcional do sistema	5 pontos
Capacidade de responder perguntas técnicas	4 pontos
Participação equilibrada dos integrantes	2 pontos
Total	30 pontos

Durante a apresentação, o professor poderá solicitar que a equipe:

- explique uma função específica do código;
- execute uma operação no sistema;
- mostre como um livro é inserido na ABB;
- explique o funcionamento da fila de reservas;
- justifique a escolha das estruturas;
- discuta a complexidade de uma operação;
- altere ou simule uma entrada durante a execução.

14 Cronograma de Desenvolvimento

O trabalho terá duração de **1 mês e 5 dias (com data da entrega 17/06/2026)**.
Segue uma sugestão:

Semana	Atividades esperadas
Semana 1	Leitura do enunciado, definição da equipe, modelagem dos TADs e planejamento da solução.
Semana 2	Implementação do TAD Livro, da ABB e das operações básicas de cadastro, busca e listagem.
Semana 3	Implementação da fila de reservas, lista de histórico e funcionalidades de empréstimo e devolução.
Semana 4	Testes, correções, finalização do relatório, README e preparação da apresentação.

15 Funcionalidades Extras

As funcionalidades abaixo não são obrigatórias, mas poderão ser consideradas como diferencial qualitativo na avaliação, especialmente em caso de trabalhos com desempenho semelhante:

- remoção de livro da Árvore Binária de Busca;
- salvamento dos dados em arquivo;
- carregamento dos dados ao iniciar o sistema;
- atendimento automático da fila de reservas na devolução;
- busca por título ou autor;
- listagem de livros indisponíveis;
- listagem de livros com fila de reserva;
- exibição visual simplificada da árvore;
- liberação adequada de memória antes de encerrar o programa.

16 Observações Importantes

- O trabalho deverá ser implementado em linguagem C.
- O uso de `struct`, ponteiros e alocação dinâmica é obrigatório.
- Não será permitido utilizar bibliotecas prontas para substituir as estruturas de dados solicitadas.

- Todos os integrantes devem compreender o código entregue.
- A equipe deverá estar preparada para explicar qualquer função implementada.
- Trabalhos que não compilarem terão forte penalização.
- Trabalhos copiados ou sem domínio técnico pelos integrantes poderão receber nota reduzida ou zero.
- O relatório e a apresentação devem estar coerentes com o código entregue.

17 Enunciado Resumido para os Alunos

Desenvolva um **Sistema de Gerenciamento de Biblioteca** em linguagem C, utilizando Tipos Abstratos de Dados, Árvore Binária de Busca, fila e lista encadeada.

O sistema deverá permitir cadastrar livros, buscar livros por código, listar livros em diferentes percursos da árvore, realizar empréstimos, devolver livros, controlar reservas e exibir o histórico de empréstimos.

A Árvore Binária de Busca deverá armazenar os livros pelo código. A fila deverá controlar as reservas de livros indisponíveis. A lista encadeada deverá armazenar o histórico de empréstimos realizados.

O trabalho deverá ser desenvolvido em até **1 mês** e entregue com código-fonte, relatório técnico, arquivo README e apresentação final. A apresentação terá peso de **30% da nota total**.